

Local Voice Agent

Full Technical Documentation — LiveKit + faster-whisper + Piper + local/hosted LLM voice assistant

Host	kali@192.168.0.200 (Kali Linux)
Project root	/home/kali/voice-agent
Web UI	https://192.168.0.200:8443
Generated	2026-08-16

1. Overview

This project is a fully self-hosted, LAN-accessible realtime voice assistant built on **LiveKit** (WebRTC signaling/media server). A browser client connects over HTTPS/WSS, speaks into its microphone, and receives spoken replies back in realtime — all routed through a Python agent worker that chains together speech-to-text, an LLM, and text-to-speech.

The LLM is switchable per-session between two providers, selected from the web UI:

- **Local** — `llama-server` (llama.cpp) running a local GGUF model, fully offline, no API key.
- **API** — any OpenAI-compatible chat-completions endpoint (configured for DeepSeek by default, but works with OpenAI, Groq, etc.)

1.1 Architecture

```
Browser (any device on the LAN)
|
|  HTTPS/WSS  (single origin, self-signed cert via Caddy)
v
Caddy :8443 -----
|-- path /rtc*, /rtc-validate*, /twirp/* --> livekit-server :7880  (WebRTC signaling)
|-- everything else                      --> web_server.py :8082  (static UI + /token API)
(WebRTC MEDIA (audio) goes browser <-> livekit-server directly, UDP, using --node-ip)

livekit-server :7880 (--dev, --node-ip 192.168.0.200)
|
|  auto-dispatches a job to the registered worker when a participant joins a room
v
agent.py  (LiveKit Agents worker, "dev" mode)
|-- STT --> custom_stt.FasterWhisperSTT  (faster-whisper, small.en, CPU, local)
|-- LLM  --> build_llm(provider, ...)      (see resolve_llm_config: room metadata > .env default)
|      "local" -> llama-server :8000 (OpenAI-compatible, GGUF model)
|      "api"   -> any OpenAI-compatible endpoint (DeepSeek by default)
|-- TTS  --> custom_tts.PiperTTS          (Piper CLI subprocess, local)
|-- VAD   --> silero.VAD (turn_detection="vad", fully local, no cloud calls)
v
Spoken audio + live transcript (lk.transcription text stream) published back to the room
v
Browser plays audio + renders live captions + 3D audio-reactive UI (Three.js)
```

1.2 Provider config flow

When the browser clicks "Engage", it POSTs to `/token` on `web_server.py` with the chosen provider/model/base_url/api_key. `web_server.py` creates the LiveKit room *explicitly* (via the LiveKit Room Service API) with that config JSON-encoded into the room's `metadata` field, *then* issues a join token for that room. When `agent.py`'s worker is dispatched into the room, its endpoint reads `ctx.room.metadata` and builds the matching LLM client for that session. This means the LLM choice is fully per-session and requires no server restart.

2. Host & Environment

Item	Value
Host	Kali Linux, hostname <code>ka li</code> , LAN IP <code>192.168.0.200</code>
CPU	Intel Core i5-4210U, 4 threads (no GPU — everything runs on CPU)
RAM	11 GiB
Python	3.13.14 (venv at <code>~/voice-agent/venv</code>)
livekit-server	1.13.5
llama-server (llama.cpp)	build 10192 (Debian, via apt)
caddy	2.11.4
livekit-agents	1.6.10
livekit-plugins-openai	1.6.10
livekit-plugins-silero	1.6.10
faster-whisper	1.2.1 (ctranslate2 4.8.1)
onnxruntime	1.28.0 (used by Piper/Silero)
numpy	2.5.2
python-dotenv	1.2.2
requests	2.34.2

2.1 Model files

File	Size	Purpose
<code>models/en_US-lessac-medium.onnx</code> (+ .json config)	61 MB	Piper TTS voice
<code>models/qwen2.5-0.5b-instruct-q4_k_m.gguf</code>	469 MB	Local LLM (llama-server), Qwen2.5-0.5B-Instruct, Q4_K_M quantized
<code>piper/piper/piper</code>	4.8 MB	Piper TTS CLI binary (static, Linux x86_64)
faster-whisper <code>small.en</code>	~480 MB	Downloaded to <code>~/.cache/huggingface</code> on first run (CTranslate2 format)

2.2 Ports

Port	Service	Exposure
7880	livekit-server (WebRTC signaling)	0.0.0.0 (proxied by Caddy for the public path)
8000	llama-server (local LLM, OpenAI-compatible)	0.0.0.0 (LAN-internal use)
8082	web_server.py (token API + static UI)	0.0.0.0 (proxied by Caddy)

8443	Caddy — HTTPS/WSS reverse proxy, self-signed cert	0.0.0.0 — this is the only port a browser needs
------	---	--

Only port **8443** needs to be reachable from a client browser. Everything else is internal to the host.

4. Source Code — Core Agent

4.1 agent.py

The LiveKit Agents worker entrypoint. Builds the STT/LLM/TTS pipeline per session and resolves which LLM provider to use from room metadata (set by the web UI) or the `.env` defaults.

```
/home/kali/voice-agent/agent.py
```

```
from __future__ import annotations

import json
import logging
import os

from dotenv import load_dotenv
from livekit import agents
from livekit.agents import Agent, AgentSession, JobContext, WorkerOptions, cli
from livekit.plugins import openai, silero

from custom_stt import FasterWhisperSTT
from custom_tts import PiperTTS

load_dotenv(os.path.join(os.path.dirname(__file__), ".env"))

logger = logging.getLogger("local-voice-agent")

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PIPER_BIN = os.path.join(BASE_DIR, "piper", "piper", "piper")
PIPER_MODEL = os.path.join(BASE_DIR, "models", "en_US-lessac-medium.onnx")
WHISPER_MODEL = os.environ.get("WHISPER_MODEL", "small.en")

# Local llama-server (default, fully offline)
LOCAL_LLM_MODEL = os.environ.get("LLM_MODEL", "qwen2.5-0.5b-instruct")
LOCAL_LLM_BASE_URL = os.environ.get("LLM_BASE_URL", "http://localhost:8000/v1")

# Server-wide default for the "api" provider: any OpenAI-compatible endpoint
# (DeepSeek, OpenAI itself, Groq, Together, etc. -- anything speaking the
# /v1/chat/completions wire format). Settable in .env, and overridable per
# session via room metadata, which is how the web UI's picker takes effect.
DEFAULT_LLM_PROVIDER = os.environ.get("LLM_PROVIDER", "local") # "local" | "api"
DEFAULT_API_BASE_URL = os.environ.get("API_BASE_URL", "https://api.deepseek.com/v1")
DEFAULT_API_KEY = os.environ.get("API_KEY", "")
DEFAULT_API_MODEL = os.environ.get("API_MODEL", "deepseek-chat")

def build_llm(
    provider: str,
    api_key: str | None = None,
    model: str | None = None,
    base_url: str | None = None,
):
    """Construct the LLM plugin for the given provider.

    provider="local" -> local llama-server (fully offline, no API key needed)
```

```

provider="api"      -> any OpenAI-compatible endpoint (DeepSeek by default,
                      but base_url/model can point anywhere that speaks
                      the OpenAI chat-completions wire format)

"""
if provider == "api":
    key = api_key or DEFAULT_API_KEY
    if not key:
        raise ValueError(
            "API provider selected but no API key was given "
            "(set API_KEY in .env, or pass one from the web UI)"
        )
    return openai.LLM(
        model=model or DEFAULT_API_MODEL,
        api_key=key,
        base_url=base_url or DEFAULT_API_BASE_URL,
    )

# default: local
return openai.LLM(
    model=model or LOCAL_LLM_MODEL, api_key="not-needed", base_url=LOCAL_LLM_BASE_URL
)

def resolve_llm_config(ctx: JobContext) -> tuple[str, str | None, str | None, str | None]:
    """Per-session override via room metadata (set by web_server.py's /token
    endpoint when the browser picks a provider), falling back to the .env
    server-wide default.
    """
    provider, api_key, model, base_url = DEFAULT_LLM_PROVIDER, None, None, None
    metadata = ctx.room.metadata
    if metadata:
        try:
            cfg = json.loads(metadata)
            provider = cfg.get("llm_provider", provider)
            api_key = cfg.get("llm_api_key") or None
            model = cfg.get("llm_model") or None
            base_url = cfg.get("llm_base_url") or None
        except (json.JSONDecodeError, AttributeError):
            logger.warning("could not parse room metadata as JSON: %r", metadata)
    return provider, api_key, model, base_url

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions=(
                "You are a helpful, friendly voice assistant. Keep responses "
                "concise, ideally under 2 sentences, since they will be spoken aloud."
            )
        )

def prewarm(proc: agents.JobProcess) -> None:
    proc.userdata["vad"] = silero.VAD.load()

async def entrypoint(ctx: JobContext) -> None:
    await ctx.connect()

```

```

provider, api_key, model, base_url = resolve_llm_config(ctx)
try:
    llm = build_llm(provider, api_key, model, base_url)
    logger.info(
        "using LLM provider=%s model=%s base_url=%s",
        provider,
        model or "(default)",
        base_url or "(default)",
    )
except ValueError as e:
    logger.warning("%s -- falling back to local LLM", e)
    provider = "local"
    llm = build_llm("local")

session = AgentSession(
    vad=ctx.proc.userdata["vad"],
    stt=FasterWhisperSTT(model_size=WHISPER_MODEL),
    llm=llm,
    tts=PiperTTS(piper_bin=PIPER_BIN, model_path=PIPER_MODEL),
    # VAD-based turn detection: fully local, avoids the framework's default
    # attempt to reach LiveKit Cloud's turn-detector inference endpoint.
    turn_detection="vad",
)

await session.start(room=ctx.room, agent=Assistant())

await session.generate_reply(
    instructions="Greet the user briefly and offer your assistance."
)

if __name__ == "__main__":
    cli.run_app(
        WorkerOptions(entrypoint_fnc=entrypoint, prewarm_fnc=prewarm)
    )

```

4.2 custom_stt.py

A minimal `livekit.agents.stt.STT` subclass wrapping **faster-whisper** (CTranslate2, CPU, int8) for fully local, offline speech-to-text. Non-streaming: implements `_recognize_impl`, converting the LiveKit `AudioBuffer` to WAV bytes and running Whisper on it in a thread-pool executor (so it doesn't block the event loop).

```
/home/kali/voice-agent/custom_stt.py
```

```

from __future__ import annotations

import asyncio
import io

from faster_whisper import WhisperModel
from livekit import rtc
from livekit.agents import APIConnectOptions, stt
from livekit.agents.types import NOT_GIVEN, NotGivenOr
from livekit.agents.utils import AudioBuffer, is_given

```



```

class FasterWhisperSTT(stt.STT):
    """Fully local, offline STT using faster-whisper (CTranslate2, CPU)."""

    def __init__(
        self,
        *,
        model_size: str = "small.en",
        device: str = "cpu",
        compute_type: str = "int8",
        language: str = "en",
    ) -> None:
        super().__init__(
            capabilities=stt.STTCapabilities(streaming=False, interim_results=False)
        )
        self._language = language
        self._model = WhisperModel(model_size, device=device, compute_type=compute_type)
        self._lock = asyncio.Lock()

    @property
    def model(self) -> str:
        return "faster-whisper"

    @property
    def provider(self) -> str:
        return "local"

    async def _recognize_impl(
        self,
        buffer: AudioBuffer,
        *,
        language: NotGivenOr[str] = NOT_GIVEN,
        conn_options: APICConnectOptions,
    ) -> stt.SpeechEvent:
        wav_bytes = rtc.combine_audio_frames(buffer).to_wav_bytes()
        lang = language if is_given(language) else self._language

        loop = asyncio.get_running_loop()

        def _transcribe() -> str:
            segments, _info = self._model.transcribe(
                io.BytesIO(wav_bytes),
                language=lang or None,
                beam_size=1,
                vad_filter=False,
            )
            return " ".join(seg.text.strip() for seg in segments).strip()

        async with self._lock:
            text = await loop.run_in_executor(None, _transcribe)

        return stt.SpeechEvent(
            type=stt.SpeechEventType.FINAL_TRANSCRIPT,
            alternatives=[stt.SpeechData(language=lang or "en", text=text)],
        )

```

4.3 custom_tts.py

A minimal `livekit.agents.tts.TTS` subclass wrapping the **Piper** CLI binary as a subprocess for fully local, offline text-to-speech. Feeds text via stdin, reads raw PCM audio from stdout (`--output-raw`), and pushes it to LiveKit's `AudioEmitter`.

```
/home/kali/voice-agent/custom_tts.py
```

```
from __future__ import annotations

import asyncio

from livekit.agents import tts
from livekit.agents.types import DEFAULT_API_CONNECT_OPTIONS, APIConnectOptions

NUM_CHANNELS = 1

class PiperTTS(tts.TTS):
    """Fully local, offline TTS using the Piper CLI binary (subprocess, raw PCM stdout)."""

    def __init__(
        self,
        *,
        piper_bin: str,
        model_path: str,
        sample_rate: int = 22050,
        length_scale: float = 1.0,
    ) -> None:
        super().__init__(
            capabilities=tts.TTSCapabilities(streaming=False),
            sample_rate=sample_rate,
            num_channels=NUM_CHANNELS,
        )
        self._piper_bin = piper_bin
        self._model_path = model_path
        self._length_scale = length_scale

    @property
    def model(self) -> str:
        return "piper"

    @property
    def provider(self) -> str:
        return "local"

    def synthesize(
        self, text: str, *, conn_options: APIConnectOptions = DEFAULT_API_CONNECT_OPTIONS
    ) -> tts.ChunkedStream:
        return PiperChunkedStream(tts=self, input_text=text, conn_options=conn_options)

class PiperChunkedStream(tts.ChunkedStream):
    def __init__(self, *, tts: PiperTTS, input_text: str, conn_options: APIConnectOptions) -> None:
        super().__init__(tts=tts, input_text=input_text, conn_options=conn_options)
        self._tts: PiperTTS = tts
```

```
async def _run(self, output_emitter: tts.AudioEmitter) -> None:
    proc = await asyncio.create_subprocess_exec(
        self._tts._piper_bin,
        "--model",
        self._tts._model_path,
        "--length_scale",
        str(self._tts._length_scale),
        "--output-raw",
        stdin=asyncio.subprocess.PIPE,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.DEVNULL,
    )
    text = (self.input_text.strip() or " ") + "\n"
    stdout_data, _ = await proc.communicate(text.encode("utf-8"))

    output_emitter.initialize(
        request_id=str(id(self)),
        sample_rate=self._tts.sample_rate,
        num_channels=self._tts.num_channels,
        mime_type="audio/pcm",
    )
    output_emitter.push(stdout_data)
    output_emitter.flush()
```

5. Source Code — Web Layer

5.1 web_server.py

Stdlib-only Python HTTP server (no framework). Serves the static `web/` directory and implements the `/token` endpoint (see §7 for the full API contract). This is the only place that talks to the LiveKit Room Service API to stamp per-session LLM config into room metadata before issuing a join token.

```
/home/kali/voice-agent/web_server.py
```

```
"""Minimal local web server for the voice-agent browser UI.

Serves the static frontend (./web/) and a /token endpoint that mints a
LiveKit access token for a fresh room, so the browser can join and start
a realtime voice conversation with the agent worker.

/token also lets the browser pick which LLM the agent should use for that
session: the local llama-server (default, no key needed) or a hosted API
such as DeepSeek (requires an API key). The choice is stamped into the
room's metadata (as JSON) before the token is issued, and agent.py reads
it back once it joins the room.

stdlib-only except for the livekit-api client already used elsewhere in
this project.
"""
from __future__ import annotations

import asyncio
import json
import os
import time
import uuid
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse

from livekit import api

LIVEKIT_URL_PUBLIC = os.environ.get("LIVEKIT_URL_PUBLIC", "wss://192.168.0.200:8443")
LIVEKIT_URL_LOCAL = os.environ.get("LIVEKIT_URL_LOCAL", "http://localhost:7880")
API_KEY = os.environ.get("LIVEKIT_API_KEY", "devkey")
API_SECRET = os.environ.get("LIVEKIT_API_SECRET", "secret")
WEB_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "web")
PORT = int(os.environ.get("WEB_PORT", "8082"))

ALLOWED_PROVIDERS = {"local", "api"}

def make_token(identity: str, room: str) -> str:
    grants = api.VideoGrants(room_join=True, room=room, can_publish=True, can_subscribe=True)
    return (
        api.AccessToken(API_KEY, API_SECRET)
        .with_identity(identity)
        .with_name(identity)
        .with_grants(grants)
```

```

        .to_jwt()
    )

async def create_room_with_metadata(room: str, metadata: dict) -> None:
    lk = api.LiveKitAPI(url=LIVEKIT_URL_LOCAL, api_key=API_KEY, api_secret=API_SECRET)
    try:
        await lk.room.create_room(
            api.CreateRoomRequest(name=room, metadata=json.dumps(metadata))
        )
    finally:
        await lk.aclose()

class Handler(BaseHTTPRequestHandler):
    def _send_json(self, payload: dict, status: int = 200) -> None:
        body = json.dumps(payload).encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Access-Control-Allow-Origin", "")
        self.end_headers()
        self.wfile.write(body)

    def _handle_token(self, params: dict) -> None:
        name = params.get("name") or f"user-{uuid.uuid4().hex[:6]}"
        room = params.get("room") or f"voice-agent-{int(time.time())}"
        provider = (params.get("provider") or "local").strip().lower()
        api_key = (params.get("api_key") or "").strip()
        model = (params.get("model") or "").strip()
        base_url = (params.get("base_url") or "").strip()

        if provider not in ALLOWED_PROVIDERS:
            self._send_json({"error": f"unknown provider {provider!r}"}, status=400)
            return

        if provider != "local" and not api_key:
            self._send_json(
                {"error": f"provider {provider!r} requires an api_key"}, status=400
            )
            return

        # Always stamp the UI's choice into room metadata explicitly -- including
        # "local" -- so the picker is authoritative and never silently falls
        # through to whatever the agent's own .env default happens to be.
        metadata = {"llm_provider": provider}
        if api_key:
            metadata["llm_api_key"] = api_key
        if model:
            metadata["llm_model"] = model
        if base_url:
            metadata["llm_base_url"] = base_url
        try:
            asyncio.run(create_room_with_metadata(room, metadata))
        except Exception as e: # noqa: BLE001
            self._send_json({"error": f"failed to configure room: {e}"}, status=502)
            return

        token = make_token(name, room)

```

```

self._send_json(
    {
        "url": LIVEKIT_URL_PUBLIC,
        "token": token,
        "identity": name,
        "room": room,
        "provider": provider,
    }
)

def do_POST(self) -> None: # noqa: N802
    if urlparse(self.path).path != "/token":
        self.send_error(404, "not found")
        return
    length = int(self.headers.get("Content-Length", "0") or "0")
    raw = self.rfile.read(length) if length else b"{}"
    try:
        params = json.loads(raw or b"{}")
    except json.JSONDecodeError:
        self._send_json({"error": "invalid JSON body"}, status=400)
        return
    self._handle_token(params)

def do_GET(self) -> None: # noqa: N802
    parsed = urlparse(self.path)

    if parsed.path == "/token":
        # convenience GET path for the simple/local case (no api key in a URL)
        self._handle_token({})
        return

    # static file serving
    path = parsed.path
    if path == "/":
        path = "/index.html"
    fs_path = os.path.normpath(os.path.join(WEB_DIR, path.lstrip("/")))
    if not fs_path.startswith(WEB_DIR) or not os.path.isfile(fs_path):
        self.send_error(404, "not found")
        return

    ctype = "text/html"
    if fs_path.endswith(".js"):
        ctype = "application/javascript"
    elif fs_path.endswith(".css"):
        ctype = "text/css"

    with open(fs_path, "rb") as f:
        body = f.read()
    self.send_response(200)
    self.send_header("Content-Type", ctype)
    self.send_header("Content-Length", str(len(body)))
    self.end_headers()
    self.wfile.write(body)

def log_message(self, fmt, *args) -> None: # quieter logs
    print(f"[web] {self.address_string()} {fmt % args}")

```

```
if __name__ == "__main__":
    server = ThreadingHTTPServer(("0.0.0.0", PORT), Handler)
    print(f"serving on :{PORT}, livekit public url = {LIVEKIT_URL_PUBLIC}")
    server.serve_forever()
```

5.2 Caddyfile

Reverse proxy providing self-signed HTTPS/WSS on a single origin (:8443), so a browser only ever needs to accept one certificate warning. Routes LiveKit's signaling paths (/rtc*, /rtc-validate*, /twirp/*) to livekit-server:7880 , and everything else to web_server.py:8082 .

```
/home/kali/voice-agent/Caddyfile
```

```
{
    auto_https disable_redirects
}

https://192.168.0.200:8443, https://localhost:8443 {
    tls internal

    # LiveKit's client SDK talks to <base-url>/rtc/* for signaling (websocket)
    # and /rtc-validate for pre-flight checks; everything else is our web UI.
    # Routing both through the SAME origin means the browser only ever needs
    # to accept one self-signed cert warning.
    @livekit path /rtc* /rtc-validate* /twirp/*
    reverse_proxy @livekit localhost:7880
    reverse_proxy localhost:8082
}
```

5.3 web/index.html

The single-file browser frontend. Uses the LiveKit JS client SDK (via CDN) to connect, publish the microphone, and subscribe to the agent's audio + live transcription text stream. Renders a JARVIS-style HUD with an audio-reactive 3D core (Three.js, via CDN) driven by real-time Web Audio API analysis of both the user's mic and the agent's voice track.

```
/home/kali/voice-agent/web/index.html
```

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>J.A.R.V.I.S – Local Voice Agent</title>
<link rel="icon" href="data:," />
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link href="https://fonts.googleapis.com/css2?family=Orbitron:wght@500;700;900&family=Share+Tech+Mono&display=swap" rel="stylesheet" />
<script src="https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/livekit-client@2/dist/livekit-client.umd.min.js">
</script>
<style>
:root {
```

```

--bg: #03060a;
--cyan: #5df1ff;
--cyan-bright: #baffff;
--cyan-dim: rgba(93, 241, 255, 0.35);
--amber: #ffb454;
--red: #ff5c6a;
--panel-bg: rgba(8, 20, 28, 0.55);
--panel-border: rgba(93, 241, 255, 0.28);
--text: #d7f6ff;
--muted: #6f93a0;
}
* { box-sizing: border-box; }
html, body {
  margin: 0;
  width: 100%;
  min-height: 100%;
  background: radial-gradient(ellipse at 50% 30%, #071019 0%, var(--bg) 65%, #000 100%);
  color: var(--text);
  font-family: "Share Tech Mono", monospace;
}
body {
  /* the fixed canvas/overlays behind the content stay pinned even if the
     HUD itself has to scroll on short viewports (phones) */
  overflow-x: hidden;
  overflow-y: auto;
}

#scene {
  position: fixed;
  inset: 0;
  z-index: 0;
  pointer-events: none;
}

.grid-overlay {
  position: fixed;
  inset: 0;
  z-index: 1;
  pointer-events: none;
  background-image:
    linear-gradient(rgba(93, 241, 255, 0.05) 1px, transparent 1px),
    linear-gradient(90deg, rgba(93, 241, 255, 0.05) 1px, transparent 1px);
  background-size: 42px 42px;
  mask-image: radial-gradient(ellipse at 50% 40%, black 0%, transparent 72%);
}

.scanlines {
  position: fixed;
  inset: 0;
  z-index: 2;
  pointer-events: none;
  background: repeating-linear-gradient(
    to bottom,
    rgba(255, 255, 255, 0.015) 0px,
    rgba(255, 255, 255, 0.015) 1px,
    transparent 2px,
    transparent 3px
  );
};

```



```

    opacity: 0.6;
}

.hud {
  position: relative;
  z-index: 3;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  padding: 22px 28px 20px;
}

.hud-top {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
}

.brand {
  font-family: "Orbitron", sans-serif;
  font-weight: 900;
  font-size: 22px;
  letter-spacing: 0.12em;
  color: var(--cyan-bright);
  text-shadow: 0 0 12px var(--cyan-dim), 0 0 28px rgba(93, 241, 255, 0.25);
}

.brand-sub {
  display: block;
  font-family: "Share Tech Mono", monospace;
  font-weight: 400;
  font-size: 10.5px;
  letter-spacing: 0.28em;
  color: var(--muted);
  margin-top: 4px;
}

.status-pill {
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 11.5px;
  letter-spacing: 0.14em;
  padding: 8px 16px;
  border: 1px solid var(--panel-border);
  border-radius: 999px;
  background: var(--panel-bg);
  backdrop-filter: blur(6px);
}

.dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: var(--muted);
  box-shadow: 0 0 6px var(--muted);
}

.dot.connecting { background: var(--amber); box-shadow: 0 0 8px var(--amber); animation: blink
0.9s infinite; }
.dot.listening, .dot.speaking { background: var(--cyan); box-shadow: 0 0 10px var(--cyan); }
.dot.error { background: var(--red); box-shadow: 0 0 8px var(--red); }
@keyframes blink { 50% { opacity: 0.25; } }

```

```
.clock {
  position: absolute;
  top: 22px;
  left: 50%;
  transform: translateX(-50%);
  font-size: 11px;
  letter-spacing: 0.2em;
  color: var(--muted);
}

.orb-stage {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: center;
  position: relative;
  min-height: 0;
}

.reticle {
  position: relative;
  width: min(46vh, 46vw, 380px);
  height: min(46vh, 46vw, 380px);
}

.corner {
  position: absolute;
  width: 26px;
  height: 26px;
  border: 1.5px solid var(--cyan-dim);
  opacity: 0.7;
}

.corner.tl { top: -14px; left: -14px; border-right: none; border-bottom: none; }
.corner.tr { top: -14px; right: -14px; border-left: none; border-bottom: none; }
.corner.bl { bottom: -14px; left: -14px; border-right: none; border-top: none; }
.corner.br { bottom: -14px; right: -14px; border-left: none; border-top: none; }

.orb-caption {
  position: absolute;
  bottom: -34px;
  left: 50%;
  transform: translateX(-50%);
  font-size: 11px;
  letter-spacing: 0.32em;
  color: var(--muted);
  white-space: nowrap;
}

.panel {
  background: var(--panel-bg);
  border: 1px solid var(--panel-border);
  backdrop-filter: blur(10px);
  box-shadow: 0 0 24px rgba(0, 0, 0, 0.35), inset 0 0 40px rgba(93, 241, 255, 0.03);
  clip-path: polygon(
    14px 0, 100% 0, 100% calc(100% - 14px), calc(100% - 14px) 100%, 0 100%, 0 14px
  );
}

.panel-label {
```

```
font-size: 10px;
letter-spacing: 0.3em;
color: var(--cyan);
opacity: 0.7;
padding: 10px 16px 0;
}

.lower {
display: flex;
gap: 14px;
max-width: 960px;
width: 100%;
margin: 14px auto 0;
align-items: stretch;
}

.transcript-panel {
flex: 1.3;
min-width: 0;
display: flex;
flex-direction: column;
}

.transcript {
padding: 8px 16px 14px;
overflow-y: auto;
max-height: 160px;
display: flex;
flex-direction: column;
gap: 7px;
}

.transcript::-webkit-scrollbar { width: 6px; }
.transcript::-webkit-scrollbar-thumb { background: var(--panel-border); border-radius: 3px; }

.line {
font-size: 12.5px;
line-height: 1.45;
}

.line .who {
color: var(--muted);
letter-spacing: 0.12em;
margin-right: 8px;
}

.line.agent .who { color: var(--cyan); }
.line.user .who { color: var(--amber); }
.line.agent { color: var(--cyan-bright); text-shadow: 0 0 6px rgba(93, 241, 255, 0.25); }
.line.user { color: var(--text); }

.empty-hint {
color: var(--muted);
font-size: 12px;
padding: 4px 0;
}

.control-panel {
flex: 1;
min-width: 300px;
padding: 12px 16px 14px;
display: flex;
flex-direction: column;
gap: 9px;
}
```

```

}
.settings-row { display: flex; gap: 8px; }
select, input[type="password"], input[type="text"] {
  flex: 1;
  background: rgba(3, 10, 14, 0.7);
  border: 1px solid var(--panel-border);
  color: var(--text);
  border-radius: 3px;
  padding: 8px 9px;
  font-size: 12px;
  font-family: "Share Tech Mono", monospace;
  outline: none;
}
select:focus, input:focus { border-color: var(--cyan); box-shadow: 0 0 8px rgba(93, 241, 255, 0.25); }
.hint {
  color: var(--muted);
  font-size: 10.5px;
  letter-spacing: 0.02em;
}

.controls { display: flex; gap: 8px; margin-top: 2px; }
button {
  appearance: none;
  flex: 1;
  background: transparent;
  border: 1px solid var(--panel-border);
  color: var(--cyan-bright);
  border-radius: 3px;
  padding: 10px 14px;
  font-family: "Share Tech Mono", monospace;
  font-size: 11.5px;
  letter-spacing: 0.14em;
  text-transform: uppercase;
  cursor: pointer;
  transition: all 120ms ease;
}
button:hover:not(:disabled) {
  border-color: var(--cyan);
  box-shadow: 0 0 14px rgba(93, 241, 255, 0.28);
  background: rgba(93, 241, 255, 0.06);
}
button:active:not(:disabled) { transform: scale(0.97); }
button:disabled { opacity: 0.3; cursor: not-allowed; }
#connectBtn { border-color: var(--cyan); color: var(--bg); background: var(--cyan); font-weight: 700; }
#connectBtn:hover:not(:disabled) { box-shadow: 0 0 20px var(--cyan); }
#connectBtn:disabled { background: transparent; color: var(--cyan-bright); }

@media (max-width: 720px) {
  .lower { flex-direction: column; }
  .brand { font-size: 17px; }
}

@media (max-width: 520px), (max-height: 700px) {
  .hud { padding: 14px 14px 16px; }
  .brand { font-size: 15px; }
  .brand-sub { font-size: 9px; letter-spacing: 0.2em; }
}

```

```

.status-pill { font-size: 10px; padding: 6px 12px; }
.clock { display: none; }
.orb-stage { flex: 0 0 auto; padding: 8px 0; }
.reticle { width: min(38vh, 62vw, 260px); height: min(38vh, 62vw, 260px); }
.orb-caption { bottom: -26px; font-size: 10px; }
.lower { margin-top: 10px; gap: 10px; }
.transcript { max-height: 110px; }
.control-panel { min-width: 0; padding: 10px 12px 12px; }
  button { padding: 12px 10px; font-size: 11px; }
}
</style>
</head>
<body>
  <canvas id="scene"></canvas>
  <div class="grid-overlay"></div>
  <div class="scanlines"></div>

  <div class="hud">
    <div class="hud-top">
      <div class="brand">J.A.R.V.I.S<span class="brand-sub">LOCAL VOICE INTELLIGENCE // OFFLINE-
CAPABLE</span></div>
      <div class="status-pill"><span class="dot" id="statusDot"></span><span
id="statusText">STANDBY</span></div>
    </div>

    <div class="clock" id="clock">00:00:00</div>

    <div class="orb-stage">
      <div class="reticle">
        <span class="corner tl"></span><span class="corner tr"></span>
        <span class="corner bl"></span><span class="corner br"></span>
        <span class="orb-caption" id="orbCaption">SYSTEM IDLE</span>
      </div>
    </div>

    <div class="lower">
      <div class="panel transcript-panel">
        <div class="panel-label">COMM LOG</div>
        <div class="transcript" id="transcript">
          <div class="empty-hint">// awaiting connection...</div>
        </div>
      </div>

      <div class="panel control-panel">
        <div class="settings-row">
          <select id="providerSelect">
            <option value="local">LOCAL LLM (OFFLINE)</option>
            <option value="api" selected>OPENAI-COMPATIBLE API</option>
          </select>
        </div>
        <div class="settings-row" id="apiFields">
          <input id="baseUrlInput" type="text" placeholder="Base URL"
value="https://api.deepseek.com" />
          <input id="modelInput" type="text" placeholder="Model" value="deepseek-v4-flash" />
        </div>
        <div class="settings-row" id="apiKeyRow">
          <input
id="apiKeyInput"

```

```

        type="password"
        placeholder="API key (sk-...)"
        value="sk-33ed9ee8f2704d1391ff0390cce78dc8"
    />
</div>
<div class="hint" id="providerHint">Speech still runs locally; only text goes to the API.
</div>
<div class="controls">
    <button id="connectBtn">Engage</button>
    <button id="muteBtn" disabled>Mute</button>
    <button id="disconnectBtn" disabled>Disconnect</button>
</div>
</div>
</div>
</div>

<audio id="remoteAudio" autoplay></audio>

<script>
    /* ----- */
    /* 3D core — Three.js audio-reactive orb */
    /* ----- */
    const canvas = document.getElementById("scene");
    const renderer = new THREE.WebGLRenderer({ canvas, antialias: true, alpha: true });
    renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
    renderer.setSize(window.innerWidth, window.innerHeight);

    const scene = new THREE.Scene();
    const camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1,
100);
    camera.position.set(0, 0, 5.2);

    const COLORS = {
        idle: new THREE.Color(0x2a5560),
        connecting: new THREE.Color(0xffb454),
        listening: new THREE.Color(0x5df1ff),
        speaking: new THREE.Color(0xbaffff),
        error: new THREE.Color(0xff5c6a),
    };

    const core = new THREE.Mesh(
        new THREE.IcosahedronGeometry(1.25, 1),
        new THREE.MeshBasicMaterial({ color: COLORS.idle, wireframe: true, transparent: true,
opacity: 0.85 })
    );
    scene.add(core);

    const glow = new THREE.Mesh(
        new THREE.SphereGeometry(0.85, 32, 32),
        new THREE.MeshBasicMaterial({
            color: COLORS.idle,
            transparent: true,
            opacity: 0.35,
            blending: THREE.AdditiveBlending,
        })
    );
    scene.add(glow);

```

```

const rings = [];
const ringDefs = [
  { r: 1.9, tube: 0.008, tilt: [Math.PI / 2.3, 0, 0], speed: 0.006 },
  { r: 2.15, tube: 0.006, tilt: [Math.PI / 3.1, Math.PI / 4, 0], speed: -0.004 },
  { r: 2.4, tube: 0.005, tilt: [Math.PI / 5, -Math.PI / 3, 0], speed: 0.0025 },
];
for (const def of ringDefs) {
  const ring = new THREE.Mesh(
    new THREE.TorusGeometry(def.r, def.tube, 8, 128),
    new THREE.MeshBasicMaterial({ color: COLORS.idle, transparent: true, opacity: 0.5 })
  );
  ring.rotation.set(...def.tilt);
  ring.userData.speed = def.speed;
  scene.add(ring);
  rings.push(ring);
}

const PARTICLE_COUNT = 260;
const particlePositions = new Float32Array(PARTICLE_COUNT * 3);
for (let i = 0; i < PARTICLE_COUNT; i++) {
  const r = 2.7 + Math.random() * 0.9;
  const theta = Math.random() * Math.PI * 2;
  const phi = Math.acos(2 * Math.random() - 1);
  particlePositions[i * 3] = r * Math.sin(phi) * Math.cos(theta);
  particlePositions[i * 3 + 1] = r * Math.sin(phi) * Math.sin(theta);
  particlePositions[i * 3 + 2] = r * Math.cos(phi);
}
const particleGeo = new THREE.BufferGeometry();
particleGeo.setAttribute("position", new THREE.BufferAttribute(particlePositions, 3));
const particles = new THREE.Points(
  particleGeo,
  new THREE.PointsMaterial({
    color: COLORS.idle,
    size: 0.02,
    transparent: true,
    opacity: 0.55,
    blending: THREE.AdditiveBlending,
  })
);
scene.add(particles);

let currentState = "idle";
let audioLevel = 0; // smoothed 0..1
let targetColor = COLORS.idle.clone();
const tmpColor = new THREE.Color();

function setVisualState(state) {
  currentState = state;
  targetColor = COLORS[state] || COLORS.idle;
}

const clockT = new THREE.Clock();
function animate() {
  requestAnimationFrame(animate);
  const t = clockT.getElapsedTime();

  // breathing baseline + audio-reactive boost
  const breathing = 0.5 + 0.5 * Math.sin(t * (currentState === "connecting" ? 3.2 : 1.1));

```

```

const level = currentState === "speaking" || currentState === "listening" ? audioLevel : 0;
const drive = Math.max(breathing * 0.25, level);

core.rotation.y += 0.0022 + level * 0.01;
core.rotation.x += 0.0009;
const coreScale = 1 + drive * 0.18;
core.scale.setScalar(coreScale);

glow.scale.setScalar(1 + drive * 0.55);
glow.material.opacity = 0.25 + drive * 0.45;

for (const ring of rings) {
  ring.rotation.z += ring.userData.speed * (1 + level * 4);
  ring.material.opacity = 0.35 + drive * 0.35;
}

particles.rotation.y += 0.0006 + level * 0.002;
particles.rotation.x += 0.0002;
particles.material.size = 0.018 + level * 0.02;

// smooth color lerp toward target
core.material.color.lerp(targetColor, 0.06);
glow.material.color.lerp(targetColor, 0.06);
particles.material.color.lerp(targetColor, 0.06);
for (const ring of rings) ring.material.color.lerp(targetColor, 0.06);

renderer.render(scene, camera);
}
animate();

window.addEventListener("resize", () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});

/* live clock, purely decorative */
const clockEl = document.getElementById("clock");
setInterval(() => {
  clockEl.textContent = new Date().toLocaleTimeString("en-GB");
}, 1000);

/* ----- */
/* Web Audio analysers (drive audioLevel from real mic / agent audio) */
/* ----- */
let audioCtx = null;
let agentAnalyser = null;
let micAnalyser = null;
let activeAnalyser = null;

function ensureAudioCtx() {
  if (!audioCtx) audioCtx = new (window.AudioContext || window.webkitAudioContext)();
  return audioCtx;
}

function makeAnalyser(mediaStreamTrack) {
  const ctx = ensureAudioCtx();
  const stream = new MediaStream([mediaStreamTrack]);

```



```

const source = ctx.createMediaStreamSource(stream);
const analyser = ctx.createAnalyser();
analyser.fftSize = 256;
analyser.smoothingTimeConstant = 0.75;
source.connect(analyser);
return analyser;
}

function sampleLevel(analyser) {
  const data = new Uint8Array(analyser.frequencyBinCount);
  analyser.getByteFrequencyData(data);
  let sum = 0;
  for (let i = 0; i < data.length; i++) sum += data[i];
  return Math.min(1, (sum / data.length / 255) * 2.2);
}

setInterval(() => {
  if (activeAnalyser) {
    audioLevel = audioLevel * 0.6 + sampleLevel(activeAnalyser) * 0.4;
  } else {
    audioLevel = audioLevel * 0.85;
  }
}, 45);

/* ----- */
/* LiveKit connection logic */
/* ----- */
const { Room, RoomEvent, Track } = LivekitClient;

const connectBtn = document.getElementById("connectBtn");
const disconnectBtn = document.getElementById("disconnectBtn");
const muteBtn = document.getElementById("muteBtn");
const statusDot = document.getElementById("statusDot");
const statusText = document.getElementById("statusText");
const orbCaption = document.getElementById("orbCaption");
const transcriptEl = document.getElementById("transcript");
const remoteAudio = document.getElementById("remoteAudio");
const providerSelect = document.getElementById("providerSelect");
const apiFields = document.getElementById("apiFields");
const apiKeyRow = document.getElementById("apiKeyRow");
const baseUrlInput = document.getElementById("baseUrlInput");
const modelInput = document.getElementById("modelInput");
const apiKeyInput = document.getElementById("apiKeyInput");
const providerHint = document.getElementById("providerHint");
const apiInputs = [baseUrlInput, modelInput, apiKeyInput];

let room = null;
let muted = false;

const PROVIDER_HINTS = {
  local: "Runs entirely offline on this machine – no data leaves the box.",
  api: "Speech still runs locally; only the text goes to the API endpoint below.",
};
const CAPTIONS = {
  idle: "SYSTEM IDLE",
  connecting: "ESTABLISHING LINK...",
  listening: "LISTENING",
  speaking: "RESPONDING",

```

```

    error: "LINK FAILURE",
  };

  providerSelect.addEventListener("change", () => {
    const isApi = providerSelect.value === "api";
    apiFields.style.display = isApi ? "flex" : "none";
    apiKeyRow.style.display = isApi ? "flex" : "none";
    providerHint.textContent = PROVIDER_HINTS[providerSelect.value];
  });

  function setStatus(kind, text, caption) {
    statusDot.className = "dot " + kind;
    statusText.textContent = text;
    setVisualState(kind || "idle");
    if (caption) orbCaption.textContent = caption;
    else orbCaption.textContent = CAPTIONS[kind] || CAPTIONS.idle;
  }

  function addLine(who, text) {
    const hint = transcriptEl.querySelector(".empty-hint");
    if (hint) hint.remove();
    const div = document.createElement("div");
    div.className = "line " + (who === "agent" ? "agent" : "user");
    const stamp = new Date().toLocaleTimeString("en-GB", { hour12: false });
    div.innerHTML = `[${stamp}] ${who}</span>${text}`;
    transcriptEl.appendChild(div);
    transcriptEl.scrollTop = transcriptEl.scrollHeight;
    return div;
  }

  async function connect() {
    const provider = providerSelect.value;
    const apiKey = apiKeyInput.value.trim();
    const baseUrl = baseUrlInput.value.trim();
    const model = modelInput.value.trim();
    if (provider === "api" && !apiKey) {
      setStatus("error", "NO API KEY");
      return;
    }

    connectBtn.disabled = true;
    providerSelect.disabled = true;
    apiInputs.forEach((el) => (el.disabled = true));
    setStatus("connecting", "CONNECTING...");
    try {
      ensureAudioCtx();
      if (audioCtx.state === "suspended") await audioCtx.resume();

      const resp = await fetch("/token", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          name: `web-${Math.random().toString(36).slice(2, 8)}`,
          provider,
          api_key: apiKey,
          base_url: baseUrl,
          model,
        }),
      });
    }
  }

```

```

    });
    const data = await resp.json();
    if (!resp.ok) throw new Error(data.error || `token request failed (${resp.status})`);
    const { url, token } = data;

    room = new Room({ adaptiveStream: true, dynacast: true });

    room.on(RoomEvent.TrackSubscribed, (track, _pub, participant) => {
      if (track.kind === Track.Kind.Audio) {
        track.attach(remoteAudio);
        if (participant.identity.startsWith("agent")) {
          agentAnalyser = makeAnalyser(track.mediaStreamTrack);
        }
      }
    });
    room.on(RoomEvent.ActiveSpeakersChanged, (speakers) => {
      const agentSpeaking = speakers.some((p) => p.identity.startsWith("agent"));
      if (agentSpeaking && agentAnalyser) {
        activeAnalyser = agentAnalyser;
        setStatus("speaking", `CONNECTED // ${room.localParticipant.identity}`);
      } else if (room && room.state === "connected") {
        activeAnalyser = micAnalyser;
        setStatus("listening", `CONNECTED // ${room.localParticipant.identity}`);
      }
    });
    room.on(RoomEvent.Disconnected, () => onDisconnected());

    // live captions for both the user's recognized speech and the agent's replies
    room.registerTextStreamHandler("lk.transcription", async (reader, participantInfo) => {
      let lineEl = null;
      const who = participantInfo.identity.startsWith("agent") ? "AGENT" : "YOU";
      for await (const chunk of reader) {
        if (!lineEl) lineEl = addLine(who, "");
        const stamp = lineEl.querySelector(".who").textContent;
        lineEl.innerHTML = `<span class="who">${stamp}</span>${chunk}`;
        transcriptEl.scrollTop = transcriptEl.scrollHeight;
      }
    });

    await room.connect(url, token);
    await room.localParticipant.setMicrophoneEnabled(true);

    const micPub = room.localParticipant.getTrackPublication(Track.Source.Microphone);
    if (micPub && micPub.track) {
      micAnalyser = makeAnalyser(micPub.track.mediaStreamTrack);
      activeAnalyser = micAnalyser;
    }

    setStatus("listening", `CONNECTED // ${room.localParticipant.identity}`);
    disconnectBtn.disabled = false;
    muteBtn.disabled = false;
  } catch (err) {
    console.error(err);
    setStatus("error", "FAILED: " + err.message);
    connectBtn.disabled = false;
    providerSelect.disabled = false;
    apiInputs.forEach((el) => (el.disabled = false));
  }
}

```

```

}

function onDisconnected() {
  setStatus("idle", "STANDBY");
  connectBtn.disabled = false;
  disconnectBtn.disabled = true;
  muteBtn.disabled = true;
  providerSelect.disabled = false;
  apiInputs.forEach((el) => (el.disabled = false));
  muted = false;
  muteBtn.textContent = "Mute";
  activeAnalyser = null;
  agentAnalyser = null;
  micAnalyser = null;
  room = null;
}

connectBtn.addEventListener("click", connect);

disconnectBtn.addEventListener("click", async () => {
  if (room) await room.disconnect();
  onDisconnected();
});

muteBtn.addEventListener("click", async () => {
  if (!room) return;
  muted = !muted;
  await room.localParticipant.setMicrophoneEnabled(!muted);
  muteBtn.textContent = muted ? "Unmute" : "Mute";
  if (muted) activeAnalyser = null;
  else if (room.state === "connected") activeAnalyser = micAnalyser;
});

// initial UI state for the api-provider fields (matches default selection)
providerSelect.dispatchEvent(new Event("change"));
setStatus("idle", "STANDBY");
</script>
</body>
</html>

```

6. Configuration — .env

All secrets and tunables live in `/home/kali/voice-agent/.env`, loaded via `python-dotenv` in both `agent.py` and (implicitly, via the same file convention) the rest of the stack. Secret values below are partially redacted in this document — see the live file on the host for the real values.

Variable	Purpose	Current value
LIVEKIT_URL	Internal LiveKit WS URL used by server-side tooling	ws://localhost:7880
LIVEKIT_API_KEY	LiveKit dev-mode API key	devkey
LIVEKIT_API_SECRET	LiveKit dev-mode API secret	secret
HF_TOKEN / HUGGING_FACE_HUB_TOKEN	Hugging Face token (voice/model downloads)	hf_miqc...REDACTED
WHISPER_MODEL	faster-whisper model size	small.en
LLM_MODEL	Local llama-server model name	qwen2.5-0.5b-instruct
LLM_BASE_URL	Local llama-server OpenAI-compatible base URL	http://localhost:8000/v1
LLM_PROVIDER	Server-wide default provider (local or api) when no room-metadata override is present	api
API_BASE_URL	Default OpenAI-compatible base URL for the "api" provider	https://api.deepseek.com
API_KEY	Default API key for the "api" provider	sk-33ed9...REDACTED
API_MODEL	Default model name for the "api" provider	deepseek-v4-flash

`.env` is `chmod 600` on the host, but is plain text and contains a live DeepSeek API key. Do not commit it to version control or copy it off the box carelessly.

Room-metadata override (per session, set by web_server.py)

JSON stored in the LiveKit room's `metadata` field, read by `agent.py`'s `resolve_llm_config()`:

```
{
  "llm_provider": "local" | "api",
  "llm_api_key": "...",           // only present when provider != local
  "llm_model": "...",            // optional override
  "llm_base_url": "..."        // optional override
}
```

7. API Reference

7.1 Internal — web_server.py

POST /token

Mints a LiveKit join token for a new (or named) room, and — for the `api` provider — first creates the room explicitly via the LiveKit Room Service API with the LLM config stamped into its metadata.

Request body (JSON):

```
{
  "name": "web-abc123",           // optional, random if omitted
  "room": "voice-agent-...",     // optional, random if omitted
  "provider": "local" | "api",   // required (defaults to "local" if omitted)
  "api_key": "sk-...",           // required if provider == "api"
  "base_url": "https://...",    // optional, overrides the provider's default
  "model": "...",               // optional, overrides the provider's default
}
```

Response 200:

```
{
  "url": "wss://192.168.0.200:8443",
  "token": "<JWT>",
  "identity": "web-abc123",
  "room": "voice-agent-1786879006",
  "provider": "api"
}
```

Error responses: `400` unknown provider or missing `api_key`; `502` failed to create/configure the LiveKit room.

GET /token

Convenience path for the simple/local case — equivalent to POSTing `{}` (defaults to `provider: "local"`, random identity/room).

GET /*

Static file server for the `web/` directory (defaults to `index.html`).

7.2 LiveKit Room Service API (server-side, via `livekit-api` Python client)

Call	Used in	Purpose
<code>api.LiveKitAPI(url, api_key, api_secret).room.create_room(api.CreateRoomRequest(name, metadata))</code>	<code>web_server.py</code>	Pre-create the room with the chosen LLM config in its metadata,

		before the browser joins
<code>api.AccessToken(key, secret).with_identity(...).with_grants(api.VideoGrants(room_join=True, room=..., can_publish=True, can_subscribe=True)).to_jwt()</code>	<code>web_server.py</code> , <code>test_room.py</code>	Generate a signed JWT join token

7.3 LiveKit client-side signaling (browser <-> livekit-server, via livekit-client JS SDK)

Endpoint	Protocol	Purpose
<code>wss://<host>:8443 rtc/v1?access_token=...</code>	WebSocket (proxied by Caddy to <code>ws://localhost:7880</code>)	WebRTC signaling (join, offer/answer, ICE, track (un)publish)
<code>https://<host>:8443 rtc/v1/validate?access_token=...</code>	HTTPS	Pre-flight connectivity check before the WS upgrade

7.4 LLM providers (OpenAI-compatible chat-completions wire format)

Provider	Endpoint	Auth	Used via
Local (llama-server)	<code>http://localhost:8000/v1/chat/completions</code>	none (<code>api_key="not-needed"</code>)	<code>livekit.plugins.openai.LLM</code>
DeepSeek (default "api" provider)	<code>https://api.deepseek.com/chat/completions</code>	Bearer <code>API_KEY</code>	<code>livekit.plugins.openai.LLM</code>

Both are called identically by `agent.py` 's `build_llm()` via `livekit.plugins.openai.LLM(model=..., api_key=..., base_url=...)` — the plugin itself is provider-agnostic; only the base URL/key/model differ. Any other OpenAI-compatible provider (OpenAI, Groq, Together, etc.) works the same way by changing those three fields in the web UI.

7.5 Local subprocess/library "APIs"

Component	Interface	Notes
faster-whisper	Python: <code>WhisperModel(model_size, device="cpu", compute_type="int8").transcribe(audio)</code>	In-process, no network call
Piper TTS	Subprocess: <code>piper --model <voice.onnx> --output-raw</code> , text piped via stdin, raw PCM read from stdout	Spawned fresh per synthesis call in <code>PiperChunkedStream._run()</code>

8. Full Setup — Every Command, From Scratch

All commands below are run as user `kali` on the target host (`192.168.0.200`), either directly on the box or via `sshpass -p '<password>' ssh kali@192.168.0.200 "<command>"` from a controller machine.

8.1 System packages

```
# llama.cpp (provides llama-server) – available directly in Kali's apt repos
sudo apt-get install -y llama.cpp llama.cpp-tools

# Caddy (HTTPS/WSS reverse proxy)
sudo apt-get install -y caddy

# LiveKit server (official install script, installs to /usr/local/bin)
curl -sSL https://get.livekit.io | bash
```

8.2 Project folder & Python environment

```
mkdir -p ~/voice-agent
cd ~/voice-agent
python3 -m venv venv
./venv/bin/pip install --upgrade pip
./venv/bin/pip install \
    livekit-agents livekit-plugins-openai livekit-plugins-silero \
    faster-whisper numpy python-dotenv livekit-plugins-noise-cancellation requests
```

8.3 Piper TTS (binary + voice)

```
mkdir -p ~/voice-agent/piper ~/voice-agent/models
cd ~/voice-agent/piper
curl -L -o piper.tar.gz \
    https://github.com/rhasspy/piper/releases/download/2023.11.14-2/piper_linux_x86_64.tar.gz
tar xzf piper.tar.gz && rm piper.tar.gz

cd ~/voice-agent/models
curl -sL -o en_US-lessac-medium.onnx \
    'https://huggingface.co/rhasspy/piper-voices/resolve/v1.0.0/en/en_US/lessac/medium/en_US-lessac-medium.onnx'
curl -sL -o en_US-lessac-medium.onnx.json \
    'https://huggingface.co/rhasspy/piper-voices/resolve/v1.0.0/en/en_US/lessac/medium/en_US-lessac-medium.onnx.json'
```

8.4 Local LLM model (GGUF)

```
cd ~/voice-agent/models
curl -sL -o qwen2.5-0.5b-instruct-q4_k_m.gguf \
    'https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct-GGUF/resolve/main/qwen2.5-0.5b-instruct-q4_k_m.gguf'
```


8.5 Deploy the source files

Copy `agent.py`, `custom_stt.py`, `custom_tts.py`, `web_server.py`, `Caddyfile`, `start_services.sh`, `test_*.py`, `.env` (all in §4–§6 above) into `~/voice-agent/`, and `web/index.html` (§5.3) into `~/voice-agent/web/`. Then:

```
chmod 600 ~/voice-agent/.env
chmod +x ~/voice-agent/start_services.sh
```

8.6 Start everything

The easiest path — one idempotent script that brings up every service if it isn't already running:

```
cd ~/voice-agent
./start_services.sh
```

Or manually, service by service:

```
cd ~/voice-agent

# 1. LiveKit server (dev mode; --node-ip is required so LAN clients can
#   establish WebRTC media, not just signaling)
setsid nohup livekit-server --dev --bind 0.0.0.0 --node-ip 192.168.0.200 \
  > livekit-server.log 2>&1 < /dev/null &
disown

# 2. Local LLM server
setsid nohup /usr/bin/llama-server \
  -m ~/voice-agent/models/qwen2.5-0.5b-instruct-q4_k_m.gguf \
  --port 8000 --host 0.0.0.0 -c 2048 \
  > llama-server.log 2>&1 < /dev/null &
disown

# 3. Token API + static web UI
setsid nohup venv/bin/python3 web_server.py > web_server.log 2>&1 < /dev/null &
disown

# 4. Caddy (HTTPS/WSS reverse proxy)
setsid nohup caddy run --config Caddyfile --adapter caddyfile \
  > caddy.log 2>&1 < /dev/null &
disown

# 5. The LiveKit agent worker itself
setsid nohup venv/bin/python3 agent.py dev > agent_dev.log 2>&1 < /dev/null &
disown
```

8.7 Use it

On any device on the same LAN, open a browser to:

```
https://192.168.0.200:8443
```

Accept the self-signed certificate warning (one-time per browser/device), allow microphone access, pick a provider (Local or API), and click **Engage**.

8.8 Verify everything is healthy

```
curl -s http://localhost:7880 # -> OK
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8000/v1/models # -> 200
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8082/ # -> 200
pgrep -f 'caddy run' && echo caddy running
pgrep -f 'agent.py dev' && echo agent worker running
```

9. Test Scripts

Script	What it verifies	Run with
<code>test_pipeline.py</code>	Fastest smoke test: faster-whisper transcribes a sample WAV, sends it to the local llama-server, speaks the reply with Piper. No LiveKit involved.	<code>venv/bin/python3 test_pipeline.py</code>
<code>test_plugins.py</code>	Exercises the <i>actual</i> FasterWhisperSTT / openai.LLM / PiperTTS plugin classes through their real livekit-agents interfaces (<code>recognize()</code> , <code>chat()</code> , <code>synthesize().collect()</code>).	<code>venv/bin/python3 test_plugins.py</code>
<code>test_room.py</code>	Full real WebRTC round-trip: joins an actual LiveKit room as a client, triggers the live <code>agent.py dev</code> worker's auto-dispatch, and records the agent's real spoken audio reply. Requires <code>livekit-server</code> and <code>agent.py dev</code> already running.	<code>venv/bin/python3 test_room.py</code>

```
# run all three in sequence
cd ~/voice-agent
venv/bin/python3 test_pipeline.py && echo && \
venv/bin/python3 test_plugins.py && echo && \
venv/bin/python3 test_room.py
```

10. Notes & Caveats

- **Self-signed certificate:** Caddy uses its own internal CA (`tls internal`). Every browser/device needs to click through the "not secure" warning once. This is expected for a LAN dev deployment; it is not a public CA-signed cert.
- **Single origin for signaling + UI:** both the web UI and LiveKit's WebSocket signaling are served from the same `:8443` origin (via Caddy path-routing) specifically so the browser only needs to trust one certificate. Splitting them across two ports (an earlier iteration of this setup) caused silent WebSocket connection failures because browsers don't share cert-trust exceptions across ports.
- **--node-ip is required** on `livekit-server` so it advertises the correct LAN IP for WebRTC media (ICE) candidates — without it, clients on other devices can complete signaling but never receive audio.
- **DeepSeek API key** is stored in plaintext in `.env` (mode 600) and is also embedded as the default value in `web/index.html`'s API-key field — acceptable for a private single-user LAN tool, but this HTML file should not be exposed outside the LAN or committed to a public repo as-is.
- **Mobile layout:** the HUD uses `min-height` + scrollable body (not a hard `100vh` clip) specifically so all three control buttons (Engage/Mute/Disconnect) remain reachable on short/narrow phone viewports.
- **Turn detection** is explicitly set to `"vad"` (Silero VAD, local) in `agent.py`'s `AgentSession` — without this, the framework attempts a call to LiveKit Cloud's turn-detector inference endpoint by default, which fails (401, no cloud credentials) before falling back locally, adding latency.
- **CPU-only inference:** this host has no GPU. faster-whisper uses int8 CTranslate2, the GGUF LLM uses Q4_K_M quantization, and Piper is CPU-native — all chosen specifically to run acceptably fast on a 4-core laptop CPU.

Appendix A — Test & Utility Script Source

start_services.sh

Idempotent script that brings up livekit-server, llama-server, web_server.py, Caddy, and agent.py dev, checking each is already running first.

```
/home/kali/voice-agent/start_services.sh
```

```
#!/usr/bin/env bash
# Starts the full local voice-agent stack:
#   livekit-server (dev mode, node-ip set for LAN media) :7880
#   llama-server  (local LLM, OpenAI-compatible)       :8000
#   web_server.py (token API + static web UI)           :8082
#   caddy         (self-signed HTTPS/WSS reverse proxy) :8443 (UI + LiveKit signaling, same origin)
#   agent.py dev  (the LiveKit agent worker)
# faster-whisper (STT) and Piper (TTS) run in-process / as subprocesses, no server needed.
set -e
cd "$(dirname "$0")"
LAN_IP="192.168.0.200"

if ! curl -s -o /dev/null http://localhost:7880; then
    echo "starting livekit-server..."
    setsid nohup livekit-server --dev --bind 0.0.0.0 --node-ip "$LAN_IP" > livekit-server.log 2>&1 < /dev/null &
    disown
fi

if ! curl -s -o /dev/null http://localhost:8000/v1/models; then
    echo "starting llama-server..."
    setsid nohup /usr/bin/llama-server \
        -m "$(dirname "$0")/models/qwen2.5-0.5b-instruct-q4_k_m.gguf" \
        --port 8000 --host 0.0.0.0 -c 2048 \
        > llama-server.log 2>&1 < /dev/null &
    disown
fi

if ! curl -s -o /dev/null http://localhost:8082/; then
    echo "starting web_server.py (token API + UI)..."
    setsid nohup venv/bin/python3 web_server.py > web_server.log 2>&1 < /dev/null &
    disown
fi

if ! pgrep -f 'caddy run' > /dev/null; then
    echo "starting caddy (HTTPS/WSS reverse proxy)..."
    setsid nohup caddy run --config Caddyfile --adapter caddyfile > caddy.log 2>&1 < /dev/null &
    disown
fi

if ! pgrep -f 'agent.py dev' > /dev/null; then
    echo "starting agent.py worker..."
    setsid nohup venv/bin/python3 agent.py dev > agent_dev.log 2>&1 < /dev/null &
    disown
fi
```

```

sleep 3
echo "livekit-server: $(curl -s http://localhost:7880 || echo DOWN)"
echo "llama-server:  $(curl -s -o /dev/null -w '%{http_code}' http://localhost:8000/v1/models ||
echo DOWN)"
echo "web_server:     $(curl -s -o /dev/null -w '%{http_code}' http://localhost:8082/ || echo
DOWN)"
echo
echo "Open in a browser on the LAN (self-signed cert -> click through the warning,"
echo "then allow microphone access):"
echo "  https://$LAN_IP:8443"

```

test_pipeline.py

Fastest smoke test of the STT -> LLM -> TTS chain, bypassing LiveKit entirely.

/home/kali/voice-agent/test_pipeline.py

```

"""End-to-end local voice pipeline smoke test: STT -> LLM -> TTS.

Exercises the same models the real LiveKit agent (agent.py) uses, without
needing a LiveKit room / live audio I/O:
    1. faster-whisper transcribes /tmp/test.wav (Piper-generated speech)
    2. the transcript is sent to the local llama-server (OpenAI-compatible)
    3. the reply text is spoken via the Piper binary and saved to reply.wav
"""
import subprocess
import time
import wave
from pathlib import Path

import requests
from faster_whisper import WhisperModel

BASE_DIR = Path(__file__).resolve().parent
PIPER_BIN = BASE_DIR / "piper" / "piper" / "piper"
PIPER_MODEL = BASE_DIR / "models" / "en_US-lessac-medium.onnx"
LLM_URL = "http://localhost:8000/v1/chat/completions"
SAMPLE_RATE = 22050

print("=== 1) STT: faster-whisper transcribing /tmp/test.wav ===")
t0 = time.time()
model = WhisperModel("small.en", device="cpu", compute_type="int8")
segments, info = model.transcribe("/tmp/test.wav", beam_size=1)
transcript = " ".join(seg.text.strip() for seg in segments).strip()
print(f"transcript: {transcript!r} (lang={info.language}, {time.time() - t0:.2f}s)")

print("\n=== 2) LLM: querying local llama-server ===")
t0 = time.time()
resp = requests.post(
    LLM_URL,
    json={
        "model": "qwen2.5-0.5b-instruct",
        "messages": [
            {

```

```

        "role": "system",
        "content": "You are a concise voice assistant, reply in one short sentence.",
    },
    {"role": "user", "content": transcript},
],
),
timeout=60,
)
resp.raise_for_status()
reply_text = resp.json()["choices"][0]["message"]["content"].strip()
print(f"reply: {reply_text!r} ({time.time() - t0:.2f}s)")

print("\n=== 3) TTS: synthesizing reply with Piper ===")
t0 = time.time()
proc = subprocess.run(
    [str(PIPER_BIN), "--model", str(PIPER_MODEL), "--output-raw"],
    input=(reply_text + "\n").encode("utf-8"),
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
)
pcm_data = proc.stdout
out_path = BASE_DIR / "reply.wav"
with wave.open(str(out_path), "wb") as w:
    w.setnchannels(1)
    w.setsampwidth(2)
    w.setframerate(SAMPLE_RATE)
    w.writeframes(pcm_data)

duration = len(pcm_data) / 2 / SAMPLE_RATE
print(
    f"wrote {out_path} : {out_path.stat().st_size} bytes, "
    f"{duration:.2f}s audio ({time.time() - t0:.2f}s synth time)"
)

print("\n=== SUMMARY ===")
print(f"STT transcript : {transcript}")
print(f"LLM reply      : {reply_text}")
print(f"TTS output      : {out_path} ({duration:.2f}s)")

```

test_plugins.py

Exercises the real livekit-agents STT/LLM/TTS plugin classes through their actual interfaces.

```
/home/kali/voice-agent/test_plugins.py
```

```

"""Tests the actual livekit-agents plugin classes used by agent.py:
custom_stt.FasterWhisperSTT and custom_tts.PiperTTS, exercised through
their real STT/TTS interfaces (recognize() / synthesize()+collect()),
plus a sanity check that the LLM plugin (openai.LLM against llama-server)
answers through the livekit-agents LLM interface too.
"""
import asyncio
import os
import sys
import time

```

```

import wave

from livekit import rtc
from livekit.agents.types import DEFAULT_API_CONNECT_OPTIONS
from livekit.plugins import openai

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from custom_stt import FasterWhisperSTT # noqa: E402
from custom_tts import PiperTTS # noqa: E402

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PIPER_BIN = os.path.join(BASE_DIR, "piper", "piper", "piper")
PIPER_MODEL = os.path.join(BASE_DIR, "models", "en_US-lessac-medium.onnx")

def load_wav_as_frame(path: str) -> rtc.AudioFrame:
    with wave.open(path, "rb") as w:
        num_channels = w.getnchannels()
        sample_rate = w.getframerate()
        data = w.readframes(w.getnframes())
    return rtc.AudioFrame(
        data=data,
        sample_rate=sample_rate,
        num_channels=num_channels,
        samples_per_channel=len(data) // (2 * num_channels),
    )

async def test_stt() -> str:
    print("=== STT plugin: FasterWhisperSTT.recognize() ===")
    stt = FasterWhisperSTT(model_size="small.en")
    frame = load_wav_as_frame("/tmp/test.wav")
    t0 = time.time()
    event = await stt.recognize([frame], conn_options=DEFAULT_API_CONNECT_OPTIONS)
    text = event.alternatives[0].text
    print(f" type={event.type} text={text!r} ({time.time() - t0:.2f}s)")
    assert text.strip(), "STT returned empty transcript"
    assert "piper" in text.lower() or "test" in text.lower(), f"unexpected transcript: {text!r}"
    print(" OK")
    return text

async def test_llm(user_text: str) -> str:
    print("\n=== LLM plugin: openai.LLM against local llama-server ===")
    from livekit.agents import llm as llm_mod

    llm = openai.LLM(
        model="qwen2.5-0.5b-instruct",
        api_key="not-needed",
        base_url="http://localhost:8000/v1",
    )
    chat_ctx = llm_mod.ChatContext()
    chat_ctx.add_message(
        role="system", content="You are a concise voice assistant, reply in one short sentence."
    )
    chat_ctx.add_message(role="user", content=user_text)

    t0 = time.time()

```



```

reply_chunks = []
async with llm.chat(chat_ctx=chat_ctx) as stream:
    async for chunk in stream:
        if chunk.delta and chunk.delta.content:
            reply_chunks.append(chunk.delta.content)
reply = "".join(reply_chunks).strip()
print(f" reply={reply!r} ({time.time() - t0:.2f}s)")
assert reply, "LLM returned empty reply"
print(" OK")
await llm.aclose()
return reply

async def test_tts(text: str) -> None:
    print("\n=== TTS plugin: PiperTTS.synthesize().collect() ===")
    tts = PiperTTS(piper_bin=PIPER_BIN, model_path=PIPER_MODEL)
    t0 = time.time()
    stream = tts.synthesize(text)
    frame = await stream.collect()
    dur = frame.samples_per_channel / frame.sample_rate
    print(
        f" sample_rate={frame.sample_rate} channels={frame.num_channels} "
        f"duration={dur:.2f}s ({time.time() - t0:.2f}s synth)"
    )
    assert dur > 0.3, f"TTS produced suspiciously short audio: {dur:.2f}s"

    out_path = os.path.join(BASE_DIR, "reply_plugin.wav")
    with wave.open(out_path, "wb") as w:
        w.setnchannels(frame.num_channels)
        w.setsampwidth(2)
        w.setframerate(frame.sample_rate)
        w.writeframes(bytes(frame.data))
    print(f" wrote {out_path} ({os.path.getsize(out_path)} bytes)")
    print(" OK")

async def main() -> None:
    transcript = await test_stt()
    reply = await test_llm(transcript)
    await test_tts(reply)
    print("\n=== ALL PLUGIN TESTS PASSED ===")
    print(f"STT transcript : {transcript}")
    print(f"LLM reply      : {reply}")

if __name__ == "__main__":
    asyncio.run(main())

```

test_room.py

Full WebRTC round-trip test through a live LiveKit room and the running agent worker.

```
/home/kali/voice-agent/test_room.py
```

```
"""Real end-to-end test: join the actual LiveKit room as a client and verify
the running agent.py worker gets dispatched, generates a greeting via the
local LLM, synthesizes it with Piper, and publishes real audio back over
WebRTC – the one thing test_pipeline.py / test_plugins.py don't cover.
```

```
Requires: livekit-server on :7880 and `agent.py dev` already running.
```

```
"""
```

```
import asyncio
import os
import time
import wave
```

```
from livekit import api, rtc
```

```
LIVEKIT_URL = "ws://localhost:7880"
API_KEY = "devkey"
API_SECRET = "secret"
ROOM_NAME = f"test-room-{int(time.time())}"
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
OUT_PATH = os.path.join(BASE_DIR, "room_reply.wav")
```

```
TIMEOUT_S = 40
RECORD_S = 8
```

```
def make_token() -> str:
    grants = api.VideoGrants(room_join=True, room=ROOM_NAME, can_publish=True, can_subscribe=True)
    return (
        api.AccessToken(API_KEY, API_SECRET)
        .with_identity("test-client")
        .with_name("test-client")
        .with_grants(grants)
        .to_jwt()
    )
```

```
async def main() -> None:
    room = rtc.Room()
    got_track = asyncio.Event()
    frames: list[rtc.AudioFrame] = []
    audio_stream_holder: dict = {}

    @room.on("track_subscribed")
    def on_track_subscribed(track, publication, participant):
        if track.kind == rtc.TrackKind.KIND_AUDIO:
            print(f"[client] subscribed to audio track from {participant.identity}")
            stream = rtc.AudioStream(track)
            audio_stream_holder["stream"] = stream
            got_track.set()

    print(f"[client] connecting to {LIVEKIT_URL} room={ROOM_NAME} ...")
    await room.connect(LIVEKIT_URL, make_token())
    print(f"[client] connected as {room.local_participant.identity}")

    try:
        await asyncio.wait_for(got_track.wait(), timeout=TIMEOUT_S)
    except asyncio.TimeoutError:
```

```

        print("[client] FAILED: no audio track received from agent within timeout")
        await room.disconnect()
        raise SystemExit(1)

    stream = audio_stream_holder["stream"]
    print(f"[client] recording up to {RECORD_S}s of agent audio ...")
    t0 = time.time()
    async for event in stream:
        frames.append(event.frame)
        if time.time() - t0 > RECORD_S:
            break
    await stream.aclose()
    await room.disconnect()

    if not frames:
        print("[client] FAILED: track subscribed but no frames received")
        raise SystemExit(1)

    sample_rate = frames[0].sample_rate
    num_channels = frames[0].num_channels
    pcm = b"".join(bytes(f.data) for f in frames)
    with wave.open(OUT_PATH, "wb") as w:
        w.setnchannels(num_channels)
        w.setsampwidth(2)
        w.setframerate(sample_rate)
        w.writeframes(pcm)

    duration = len(pcm) / 2 / num_channels / sample_rate
    print(f"[client] wrote {OUT_PATH}: {os.path.getsize(OUT_PATH)} bytes, {duration:.2f}s audio")
    assert duration > 0.5, f"received suspiciously little audio: {duration:.2f}s"
    print("\n=== ROOM ROUND-TRIP TEST PASSED ===")

if __name__ == "__main__":
    asyncio.run(main())

```